

161.777 Practical Data Mining

Project Part 2, 2026

```
# Add any other packages you need to load here.
library(tidyverse)
library(arules)
library(cluster)
library(factoextra)
library(tidyverse)
library(skimr)
library(naniar)
library(plotly)

# Read in the data
wv_survey <- read_csv("https://www.massey.ac.nz/~jcmarsha/161777/assessment/data/wv_survey2014.csv")
flags <- read.transactions(
  "https://www.massey.ac.nz/~jcmarsha/161777/assessment/data/flags.txt",
  cols = 1
)
```

Exercise 3: Clustering the New Zealand World Values Survey

3.1 Initial exploration of the dataset

1. Plots and summary tables examining the distributions of the variables. Comment on the main patterns in the distributions.

```
# Step 1: summary tables examining the distributions of the variables
wv_survey |> skim()
```

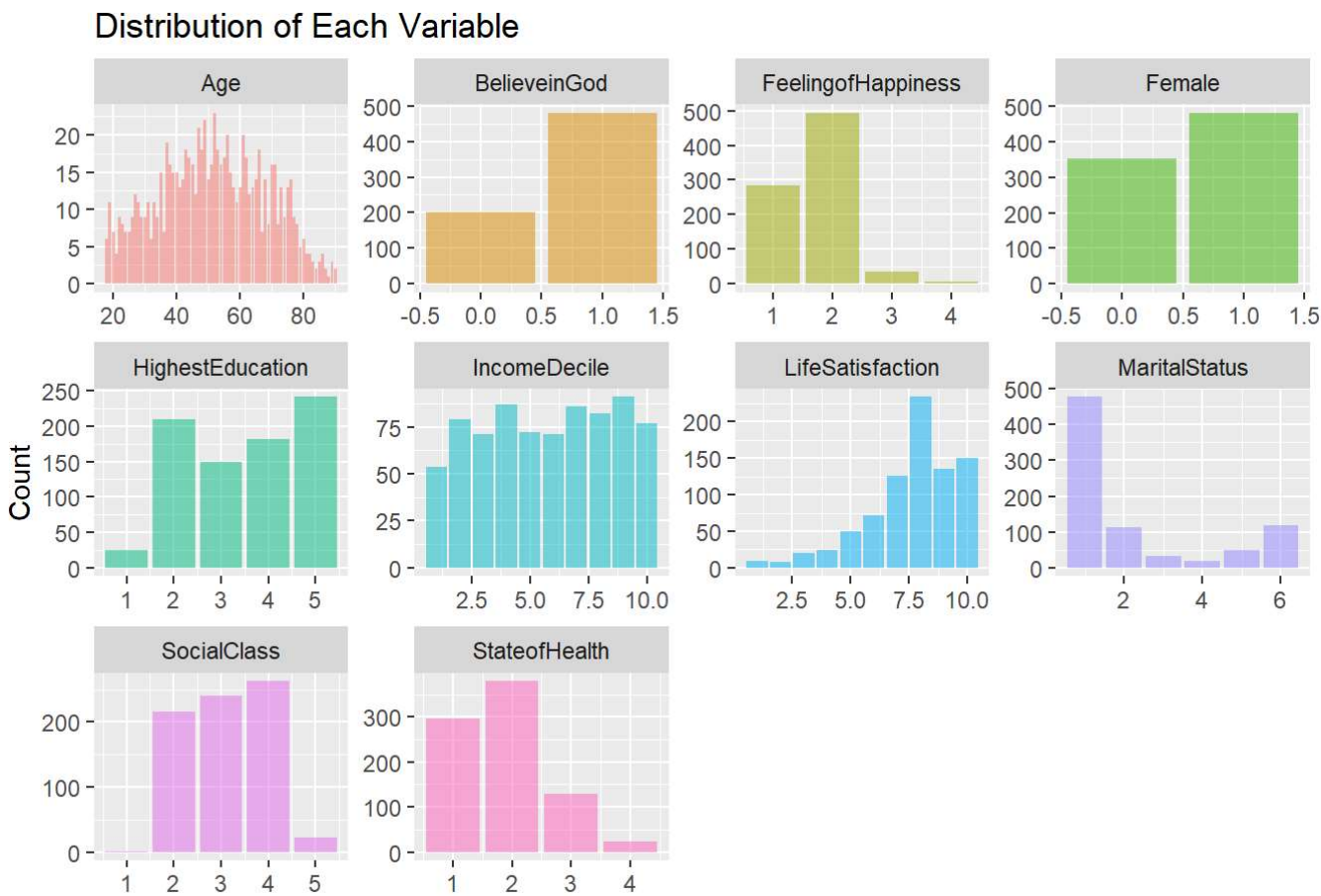
Data summary

Name	wv_survey
Number of rows	841
Number of columns	10
Column type frequency:	
numeric	10
Group variables	
None	

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Female	6	0.99	0.58	0.49	0	0	1.0	1	1	
Age	13	0.98	51.44	16.90	18	39	51.5	64	90	
StateofHealth	13	0.98	1.85	0.78	1	1	2.0	2	4	
MaritalStatus	29	0.97	2.27	1.88	1	1	1.0	3	6	
HighestEducation	34	0.96	3.50	1.25	1	2	4.0	5	5	
LifeSatisfaction	12	0.99	7.65	1.94	1	7	8.0	9	10	
IncomeDecile	71	0.92	5.72	2.81	1	3	6.0	8	10	
SocialClass	99	0.88	3.12	0.87	1	2	3.0	4	5	
BelieveinGod	160	0.81	0.71	0.46	0	0	1.0	1	1	
FeelingofHappiness	22	0.97	1.71	0.58	1	1	2.0	2	4	

```
# Step 2: Distribution of each variable
wv_survey |>
  pivot_longer(
    cols = everything(),
    names_to = "Variable",
    values_to = "Value"
  ) |>
  filter(!is.na(Value)) |>
  count(Variable, Value) |>
  ggplot(aes(x = Value, y = n, fill = Variable)) +
  geom_col(alpha = 0.5) +
  facet_wrap(vars(Variable), scales = "free") +
  guides(fill = "none") +
  labs(title = "Distribution of Each Variable",
       x = NULL, y = "Count",
       caption = "Data: World Values Survey 2014")
```

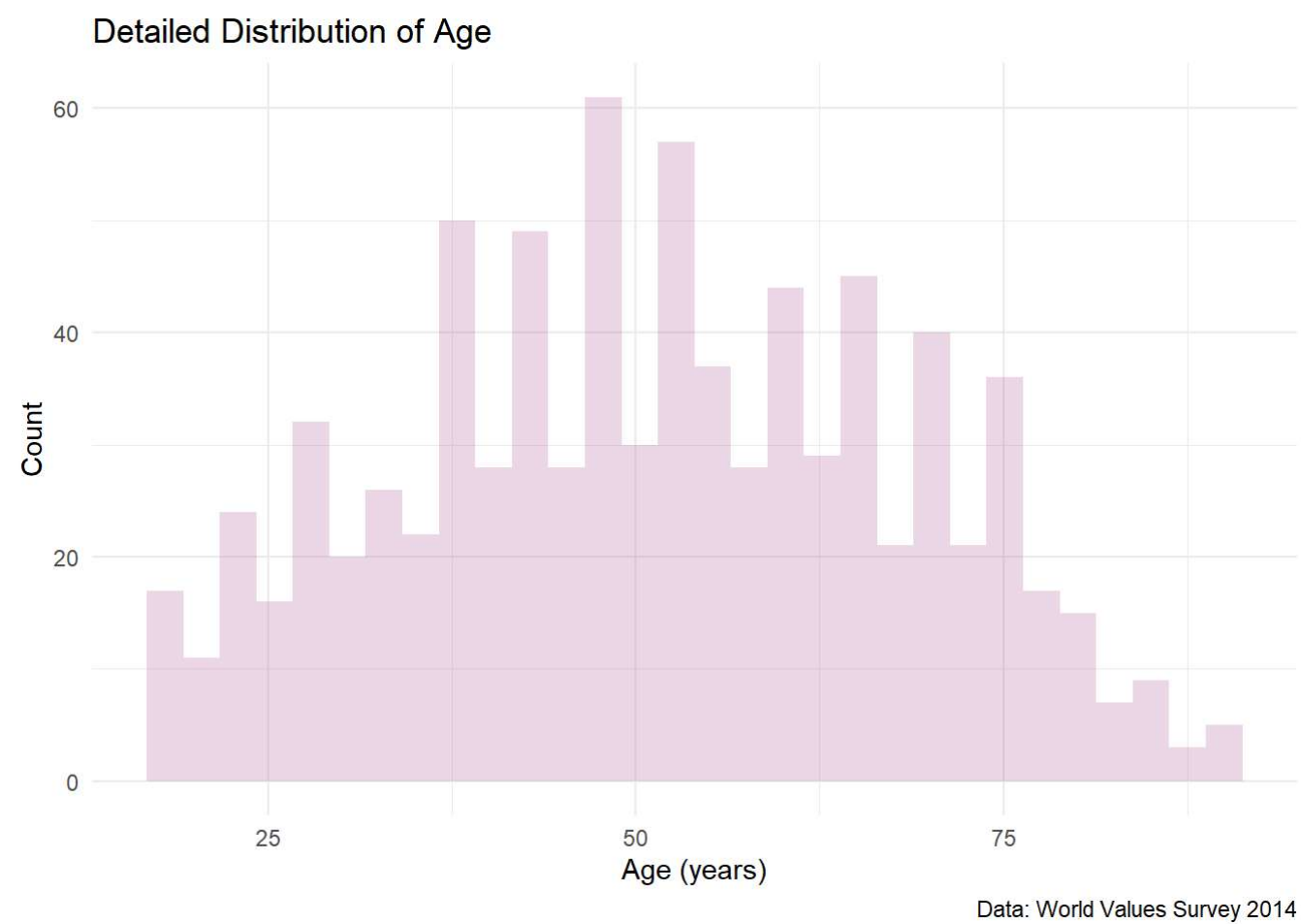


Data: World Values Survey 2014

From the summary table and the plot, we can see that:

1. Age is roughly symmetric and widely spread (18–90), centred on middle age (median ≈ 51).
 2. Female, BelieveinGod and MaritalStatus are highly unbalanced,sl ightly more females, ~70% believe in God, and “married” dominates marital status.
 3. StateofHealth and FeelingofHappiness are right-skewed (most responses at the “good” end), while LifeSatisfaction is left-skewed (concentrated at 7–10), all three lean toward positive outcomes.
 4. HighestEducation, IncomeDecile and SocialClass cluster in the middle-to-high categories, with income the most evenly spread across deciles.
2. A brief comment on how survey participation may bias the sample.

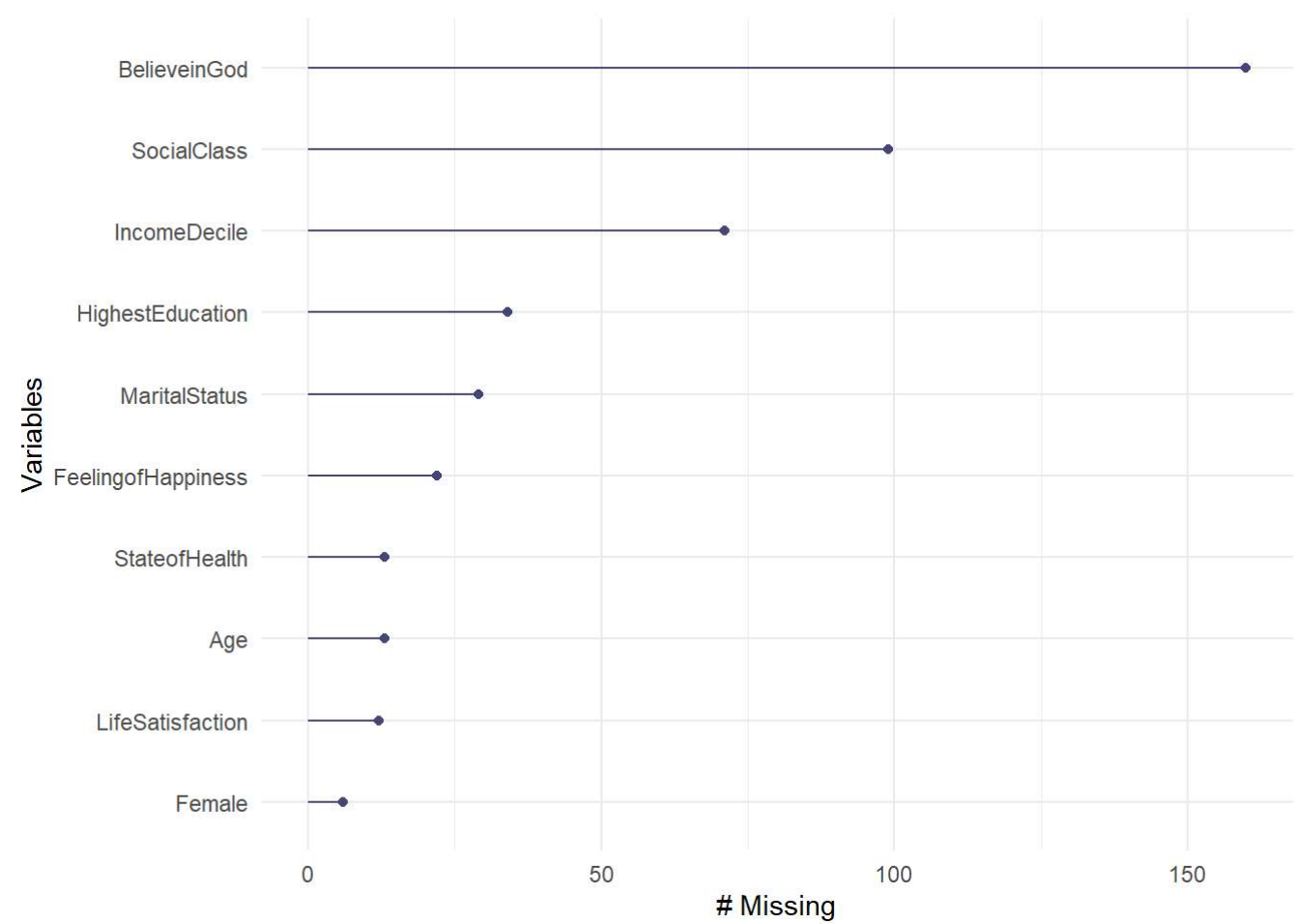
```
# Age covers a much bigger range than the other variables, so I gave it its own histogram to see the distribution more clearly.
wv_survey |>
  filter(!is.na(Age)) |>
  ggplot(aes(x = Age)) +
  geom_histogram(bins = 30, fill = "#C582B2", alpha = 0.3) +
  labs(title = "Detailed Distribution of Age",
       x = "Age (years)", y = "Count",
       caption = "Data: World Values Survey 2014") +
  theme_minimal()
```



From the distribution plots, we can see that:

1. Most respondents are married, have received some education, and fall into the middle-to-upper social classes or are employed. Since these tend to be better-off groups, this probably pushes happiness and life satisfaction toward the higher end.
 2. In contrast, people who are unmarried, less educated, of lower social class, or elderly make up only a small part of the sample, so these groups are under-represented and the sample may be biased.
 3. To get results that better reflect reality, we should first balance the sample on background variables such as gender, age, income, education, and social class, and only then examine outcomes like belief, happiness, and life satisfaction.
3. The three variables with the most missing values, with plausible reasons for this missingness.

```
# We could already see the missing values in the skim table from Step 1, but it's a bit hard to read, so here I've plotted them in order to show it more clearly.
gg_miss_var(wv_survey)
```



Based on the chart:

1. The three variables with the most missing values are BelieveinGod, SocialClass, and IncomeDecile.
2. BelieveinGod has the highest missingness. This is probably the most personal question in the entire survey. Whether or not someone believes in God is a deeply private, almost untouchable part of who they are. People tend to get quite guarded about it, so it's not surprising that so many chose not to answer.
3. SocialClass comes next. This one's tricky because the question itself is kind of judgemental by nature,nobody wants to label themselves "lower class", but claiming "upper class" feels uncomfortable too. And honestly, the boundaries are blurry. Like, how do you classify someone who earns a lot but works crazy hours versus someone who's wealthy but doesn't work at all? It's genuinely hard to answer, so people just don't.

4. Finally, IncomeDecile. A lot of people probably don’t even know what a decile is, which is already a barrier. On top of that, we tend to compare ourselves to the people around us, so if everyone in your circle earns well, you might think you’re just “average” when you’re actually in a high decile. And of course, money is always a sensitive topic, so people are naturally hesitant to share that kind of information.

3.2 Normalised dataset

Prepare a new version of the dataset using all 10 variables listed above.

```
# Step1: Define 10 variables
survey_vars <- c(
  "Female",
  "Age",
  "StateofHealth",
  "MaritalStatus",
  "HighestEducation",
  "LifeSatisfaction",
  "IncomeDecile",
  "SocialClass",
  "BelieveinGod",
  "FeelingofHappiness"
)
```

Remove rows with missing values in any of the 10 variables, then normalise all 10 variables.

```
# Step2:Remove any rows containing missing values (NA)
wv_data <- wv_survey |>
  select(all_of(survey_vars)) |>
  drop_na()

# Step3:Standardize all columns to z-scores
wv_scaled <- wv_data |>
  select(all_of(survey_vars)) |>
  mutate(across(everything(), ~ as.numeric(scale(.x))))

wv_data
```

```
## # A tibble: 516 × 10
##   Female   Age StateofHealth MaritalStatus HighestEducation LifeSatisfaction
##   <dbl> <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
## 1     1     29             2             2             3             3
## 2     1     22             1             1             4             9
## 3     1     27             2             6             4             8
## 4     0     25             4             2             4             3
## 5     0     56             1             1             5             7
## 6     1     54             2             1             5            10
## 7     0     72             1             1             5             9
## 8     1     37             1             1             5            10
## 9     1     76             2             5             2             5
## 10    0     74             2             1             2            10
## # i 506 more rows
## # i 4 more variables: IncomeDecile <dbl>, SocialClass <dbl>,
## #   BelieveinGod <dbl>, FeelingofHappiness <dbl>
```

```
wv_scaled
```

```
## # A tibble: 516 × 10
##   Female   Age StateofHealth MaritalStatus HighestEducation LifeSatisfaction
##   <dbl> <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
## 1  0.872 -1.24      0.269      -0.107      -0.499      -2.49
## 2  0.872 -1.66     -1.00     -0.646      0.326      0.678
## 3  0.872 -1.36      0.269      2.05       0.326      0.150
## 4 -1.15  -1.48      2.81      -0.107      0.326     -2.49
## 5 -1.15   0.401    -1.00     -0.646      1.15     -0.377
## 6  0.872  0.280      0.269     -0.646      1.15      1.21
## 7 -1.15   1.37     -1.00     -0.646      1.15      0.678
## 8  0.872 -0.753    -1.00     -0.646      1.15      1.21
## 9  0.872  1.62      0.269      1.51     -1.32     -1.43
## 10 -1.15   1.49      0.269     -0.646     -1.32      1.21
## # i 506 more rows
## # i 4 more variables: IncomeDecile <dbl>, SocialClass <dbl>,
## #   BelieveinGod <dbl>, FeelingofHappiness <dbl>
```

Comment on why normalisation is appropriate before cluster analysis for these data.

1. The variables in this dataset have very different ranges, Age goes from 18 to 90, LifeSatisfaction and IncomeDecile are on a 1–10 scale, and binary variables like Female and BelieveinGod only take 0 or 1.
2. Without standardisation, when calculating distances, especially Euclidean distance. Age will dominate. A variable like BelieveinGod with a range of just 0 to 1 basically gets ignored.

3. After standardisation, all variables are on the same starting line. The clustering algorithm can then pick up on similarities across all variables equally, giving us results that are actually meaningful and unbiased.

3.3 Hierarchical clustering dendrogram

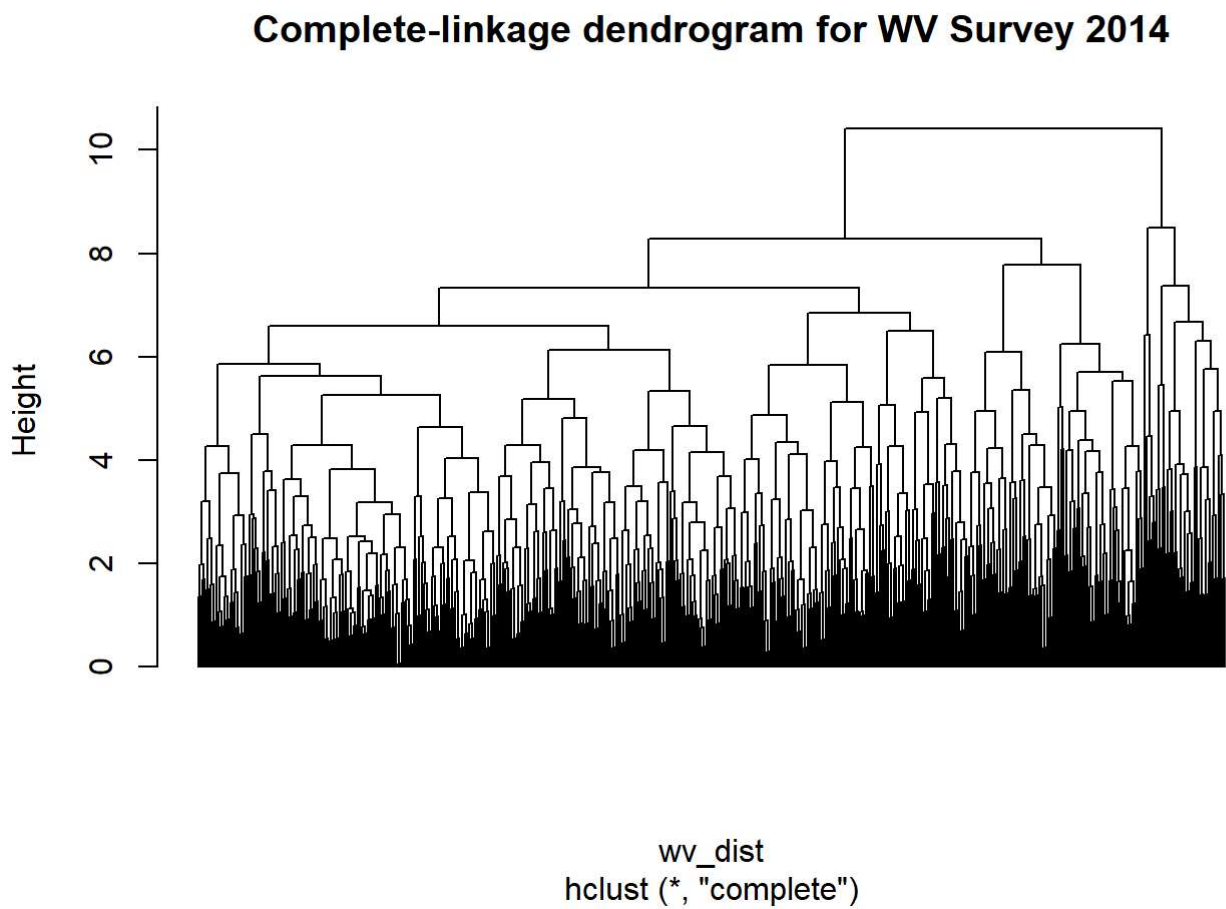
Using the complete-case, normalised 10-variable dataset from Section 3.2, produce a dendrogram of the sample.

Use Euclidean distance and complete linkage.

```
# Step 1: Compute Euclidean distance
wv_dist <- dist(wv_scaled, method = "euclidean")

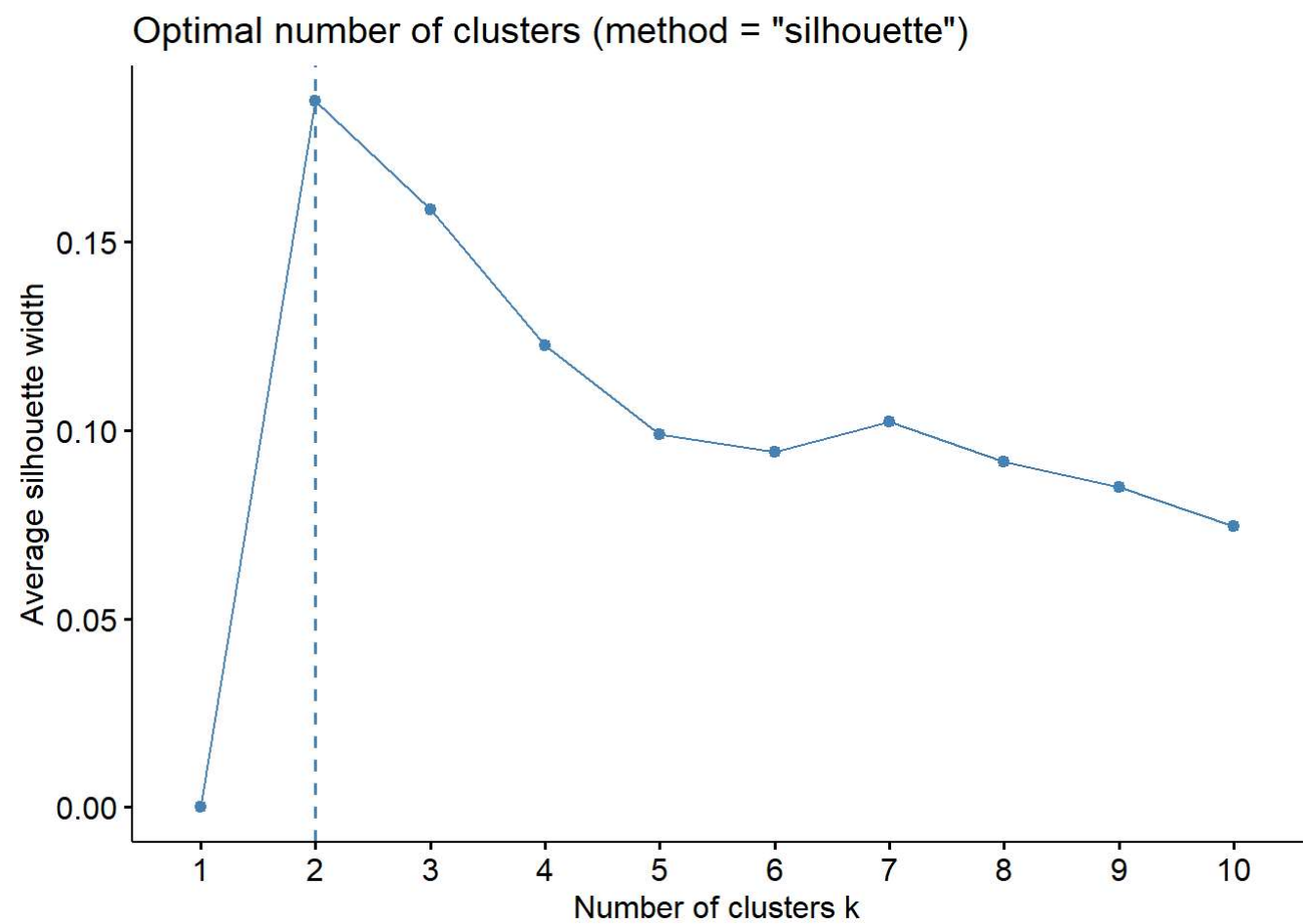
# Step 2: Hierarchical clustering with complete linkage
wv_hc <- hclust(wv_dist, method = "complete")

# Step 3: Plot dendrogram
plot(
  wv_hc,
  labels = FALSE,
  hang = -1,
  main = "Complete-linkage dendrogram for WV Survey 2014"
)
```



What do you think is the best number of clusters to divide the sample into? Briefly justify your answer.

```
# Use the silhouette measure to choose k
fviz_nbclust(
  wv_scaled,
  FUNcluster = factoextra::hcut,
  hc_method = "complete",
  k.max = 10,
  method = "silhouette"
)
```



1. From the dendrogram, we could argue for 2, 3, 4, or even 5 clusters, there’s no clear single answer just from looking at it.
2. So we use the silhouette method to make a more informed decision.
3. From the plot, $k = 2$ is the best choice, the average silhouette width peaks here, meaning the clustering is at its clearest. As k increases to 3, 4, 5, the silhouette width just keeps dropping.

3.4 k-means clustering

Perform a k-means cluster analysis of the same complete-case, normalised 10-variable dataset, based on Euclidean distances.

Choose an appropriate value of k from 1, 2, ..., 10, and explain why you chose it. Include graphs of total within-cluster sum of squares, average silhouette width, and silhouettes for your chosen solution.

```
# Step 1: Elbow plot
set.seed(123)

wv_elbow <- tibble(
  k = 1:10,
  tot_withinss = NA_real_
)

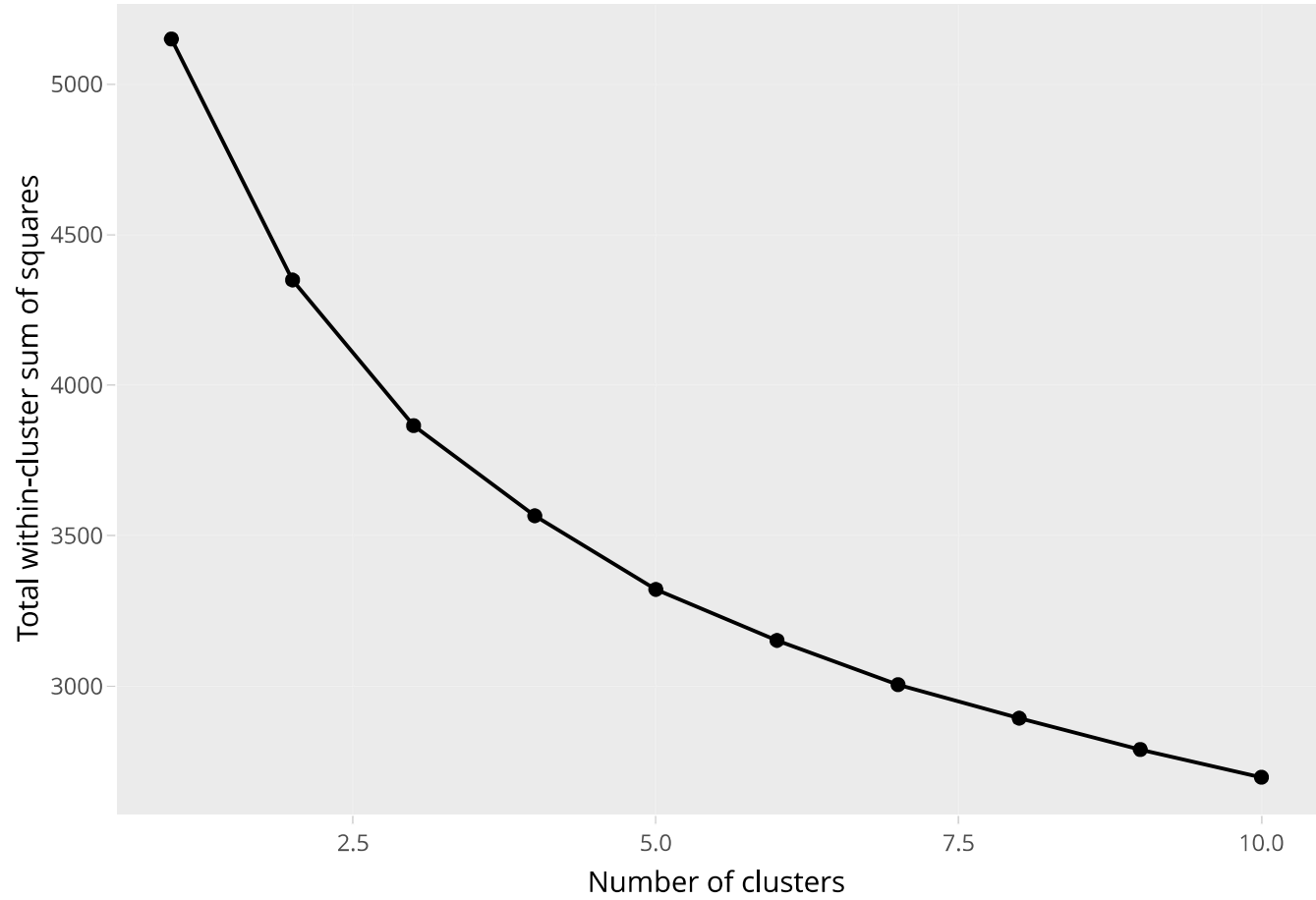
for (i in 1:nrow(wv_elbow)) {
  k_value <- wv_elbow$k[i]
  # Run K-means: Split into K clusters, try 20 times to get the best fit
  km_fit <- kmeans(wv_scaled, centers = k_value, nstart = 20)
  # Save result: Store the distance error (the smaller, the better)
  wv_elbow$tot_withinss[i] <- km_fit$tot.withinss
}

wv_elbow
```

```
## # A tibble: 10 × 2
##       k tot_withinss
##   <int>      <dbl>
## 1     1      5150.
## 2     2      4349.
## 3     3      3866.
## 4     4      3566.
## 5     5      3322.
## 6     6      3152.
## 7     7      3005.
## 8     8      2894.
## 9     9      2790.
## 10    10      2698.
```

```
# Step 1: Elbow plot
wv_elbow_plot <- ggplot(wv_elbow) +
  aes(x = k, y = tot_withinss) +
  geom_line() +
  geom_point() +
  labs(x = "Number of clusters", y = "Total within-cluster sum of squares")

ggplotly(wv_elbow_plot, tooltip = "text")
```

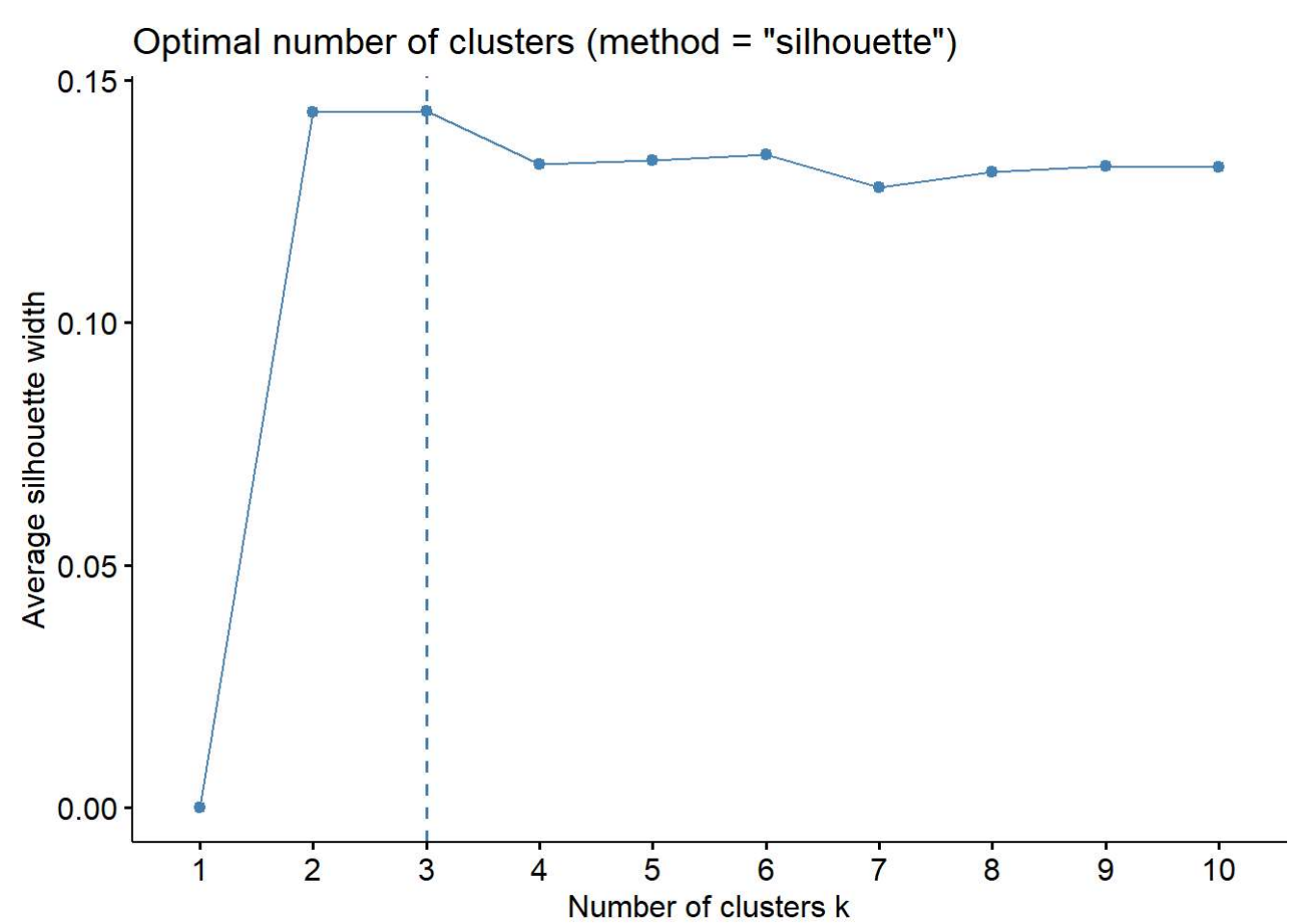
The elbow plot is a bit hard to read here, there’s no really obvious elbow point.

Looking at the curve, the biggest drop is from k=1 to k=2, then k=2 to k=3 still drops a fair bit, and after k=3 it starts to flatten out gradually.

So honestly, we could argue for either k=2 or k=3 just from this plot alone, neither is super clear cut. That’s why we need the silhouette plot to help make a more confident decision.

```
# Step 2: Silhouette width across k values for k-means
set.seed(123)

fviz_nbclust(
  wv_scaled,
  FUNcluster = kmeans,
  method = "silhouette",
  k.max = 10,
  nstart = 20
)
```

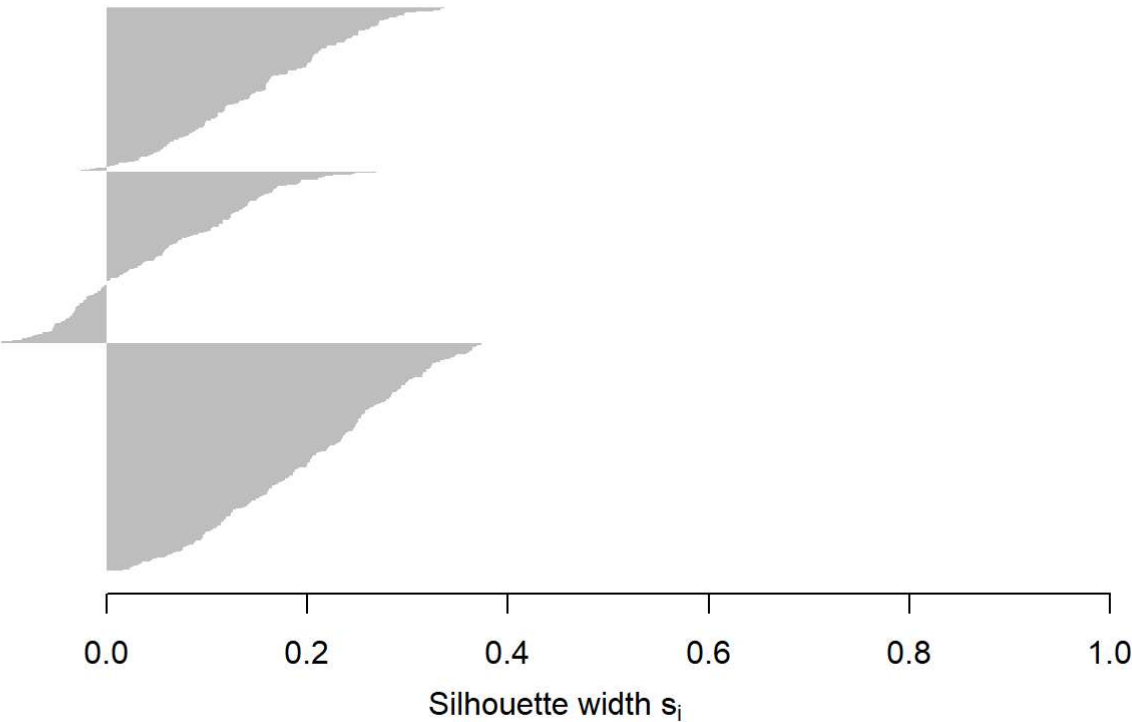


From the silhouette plot, the dashed line points to k=3, meaning k=3 has a slightly higher average silhouette width than k=2. So we’ll go with k=3 here.

```
set.seed(123)
wv_km3 <- kmeans(wv_scaled, centers = 3, nstart = 25)

# Step 3: Silhouette plot for the chosen k = 3 solution
#From help usage:silhouette ## Default S3 method:silhouette(x, dist, dmatrix, ...)
plot(silhouette(wv_km3$cluster, dist(wv_scaled)),
     border = NA, do.clus.stat = FALSE, do.n.k = FALSE)
```


Silhouette plot of (x = wv_km3\$cluster, dist = dist(wv_scaled))



From the plot:

- 1. Cluster 1 is mostly positive with only a tiny few close to zero, generally looks fine.
- 2. Cluster 2 has a small number of negative values, meaning some observations in this group probably fit better elsewhere.
- 3. Cluster 3 is the strongest, all positive, and the widest.

Overall, the clustering looks pretty reasonable.

```
set.seed(123)
# Fit a final model with K = 3 and describe clusters using their means
wv_km3 <- kmeans(wv_scaled, centers = 3, nstart = 25)

wv_km_clustered <- wv_data |>
  mutate(Cluster = factor(wv_km3$cluster))

wv_km_means <- wv_km_clustered |>
  group_by(Cluster) |>
  summarise(across(all_of(survey_vars), mean), n = n()) |>
  arrange(desc(n))

wv_km_means
```

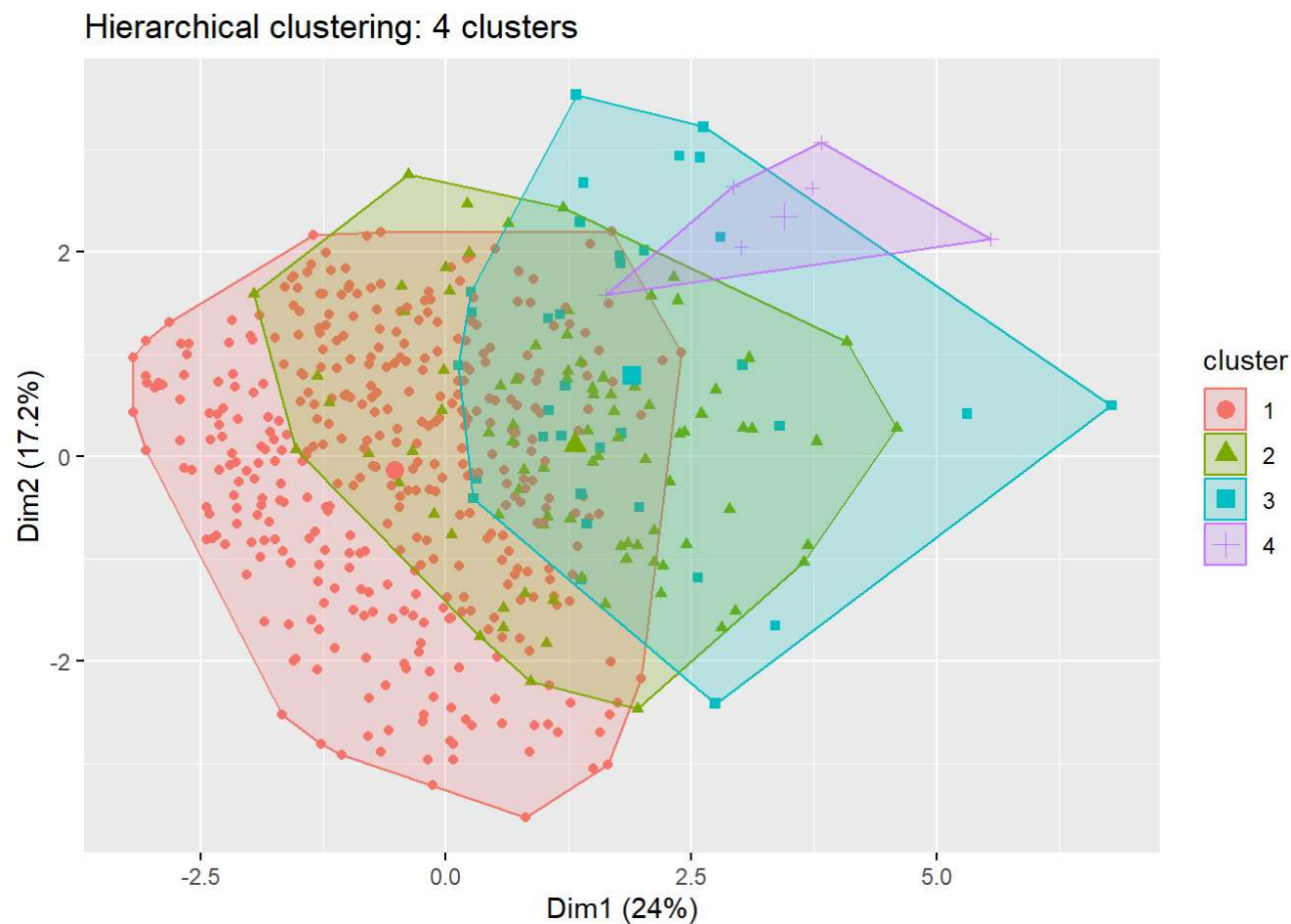
```
## # A tibble: 3 × 12
##   Cluster Female Age StateofHealth MaritalStatus HighestEducation
##   <fct>    <dbl> <dbl>         <dbl>         <dbl>         <dbl>
## 1 3      0.627  45.4         1.34         1.52         4.28
## 2 2      0.554  39.7         2.31         3.29         3.80
## 3 1      0.5    65.1         1.87         2           2.47
## # i 6 more variables: LifeSatisfaction <dbl>, IncomeDecile <dbl>,
## #   SocialClass <dbl>, BelieveinGod <dbl>, FeelingofHappiness <dbl>, n <int>
```

- 1. Cluster 3 (n = 209): Middle-aged (45), best health (1.34), highest education (4.28), highest income (7.90), highest life satisfaction (8.44) and happiness (1.44), the best-off group overall.
- 2. Cluster 2 (n = 157): Youngest (39), but worse health (2.31), middle income (5.25), lowest life satisfaction (6.08) and happiness (2.13), interesting that the youngest group is actually doing the worst across some measures.
- 3. Cluster 1 (n = 150): Oldest (65), lowest income (3.87), lowest education (2.47), yet life satisfaction is still quite high (8.41), close to Cluster 3 ,suggesting that low income doesn’t necessarily mean low life satisfaction.

3.5 Visualisation of hierarchical clustering

Regardless of your answer in Section 3.3, show a `fviz_cluster` graph based on 4 clusters from complete-linkage hierarchical clustering using the complete-case, normalised 10-variable dataset.


```
# Cut tree into 4 clusters
wv_hc_groups <- cutree(wv_hc, k = 4)
# The code follows the same logic as the usage (fviz_cluster) in the help documentation, and I double-checked the Arguments section for how object is defined.
fviz_cluster(
  object = list(data = wv_scaled, cluster = wv_hc_groups),
  data = NULL,
  geom = "point",
  ellipse = TRUE,
  ellipse.type = "convex",
  pointsize = 1.5,
  labelsize = 12,
  main = "Hierarchical clustering: 4 clusters"
)
```



Does the graph show sufficient separation between the clusters? Explain.

No, the separation is not sufficient.

1. Cluster 1 and Cluster 2 overlap heavily with each other.
2. Cluster 3 and Cluster 4 also overlap quite a bit.
3. Cluster 3 and 4 do sit slightly further right on Dim1, so there's a little bit of separation from Cluster 1, but overall all four clusters are pretty much mixed together.

3.6 Interpretation of k-means clusters

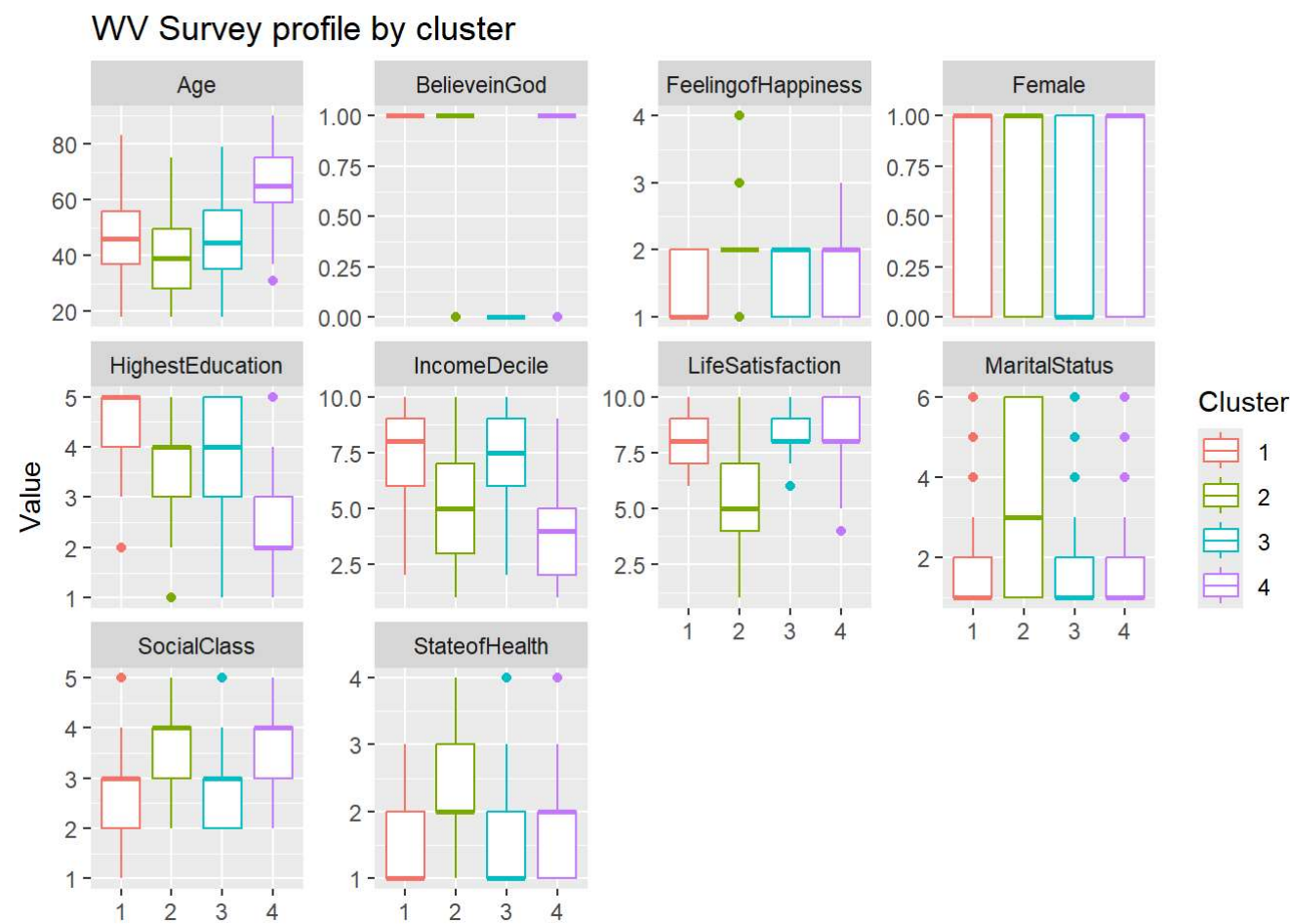
Assume 4 clusters and use k-means on the complete-case, normalised 10-variable dataset.

```
# Step 1: Fit k-means with k=4
set.seed(123)
wv_km4 <- kmeans(wv_scaled, centers = 4, nstart = 25)

wv_km4_clustered <- wv_data |>
  mutate(Cluster = factor(wv_km4$cluster))
```

Explore the differences among the clusters using boxplots or suitable summary tables. For each cluster, write one or two sentences characterising the cluster in terms of the original variables.

```
# Step 2: boxplot
wv_km4_clustered |>
  pivot_longer(
    ~Cluster,
    names_to = "Variable",
    values_to = "Value"
  ) |>
  ggplot(aes(x = Cluster, y = Value, colour = Cluster)) +
    geom_boxplot() +
    facet_wrap(vars(Variable), scales = "free_y") +
    labs(
      x = NULL,
      y = "Value",
      title = "WV Survey profile by cluster"
    )
```



1. Cluster 1 (red): Middle-aged (around 45), high education and income, all believe in God, high life satisfaction and good health, basically the high-education, high-income, religious group.
2. Cluster 2 (green): Youngest (around 40), but worst health, middle income, and clearly the lowest life satisfaction (median around 5) and happiness, the youngest group but doing the worst across most measures.
3. Cluster 3 (blue): Also middle-aged (around 45), high education and income, but no belief in God at all, pretty much identical to Cluster 1 except for religion.
4. Cluster 4 (purple): Oldest (around 65), lowest education and income, yet life satisfaction is still quite high older, lower income people who are actually pretty content with life.

```
# Step 3: Cluster means table
wv_km4_means <- wv_km4_clustered |>
  group_by(Cluster) |>
  summarise(across(all_of(survey_vars), mean), n = n()) |>
  arrange(desc(n))

wv_km4_means
```

```
## # A tibble: 4 × 12
##   Cluster Female Age StateofHealth MaritalStatus HighestEducation
##   <fct>   <dbl> <dbl>         <dbl>         <dbl>         <dbl>
## 1 1       0.708 46.1          1.41          1.65          4.31
## 2 4       0.527 65.0          1.94          1.84          2.34
## 3 3       0.434 45.5          1.5           2.02          4.04
## 4 2       0.568 40.1          2.46          3.56          3.61
## # i 6 more variables: LifeSatisfaction <dbl>, IncomeDecile <dbl>,
## #   SocialClass <dbl>, BelieveinGod <dbl>, FeelingofHappiness <dbl>, n <int>
```

Based on the cluster means table above, we can see that:

1. Cluster 1 (n = 154): Middle-aged (46), best health (1.41), highest education (4.31), highest income (7.58), all believe in God (1.00), high life satisfaction (8.22) and strongest happiness (1.48), the high-education, high-income, religiously devout group.
2. Cluster 4 (n = 129): Oldest (65), lowest education (2.34), lowest income (3.84), yet life satisfaction is still quite high (8.40), same pattern as before, older low-income people are actually pretty content.
3. Cluster 3 (n = 122): Middle-aged (46), fairly high education (4.04) and income (7.11), but no belief in God at all (0.00), very similar to Cluster 1 actually, the only real difference is religion.
4. Cluster 2 (n = 111): Youngest (40), worst health (2.46), middle income (4.75), lowest life satisfaction (5.43) and happiness (2.21), the youngest group but doing the worst across most measures.

The boxplot and the cluster means table tell the same story, the four clusters show clear and consistent differences across the key variables.

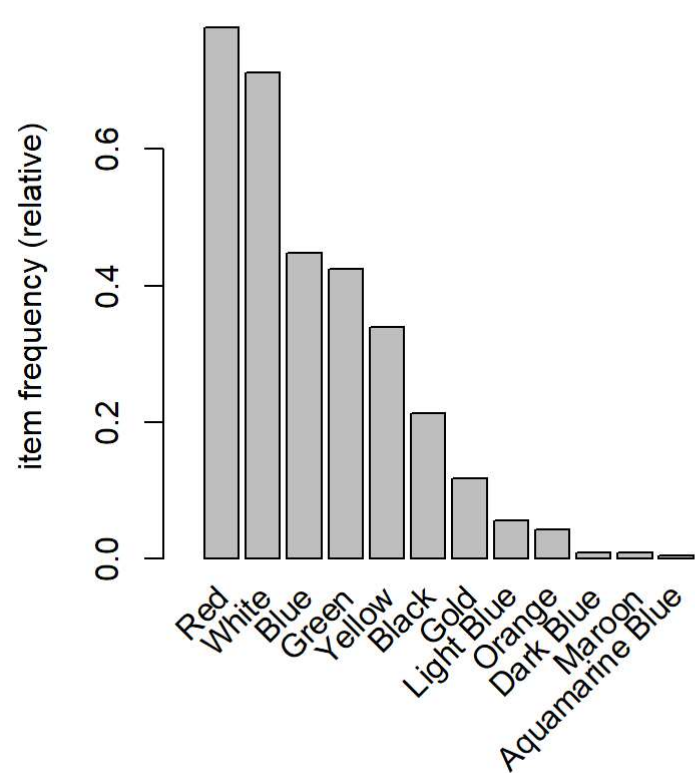
Exercise 4: Association rules for flag colours

4.1 Frequency of colour use

Make a plot showing how frequently the various colours have been used.

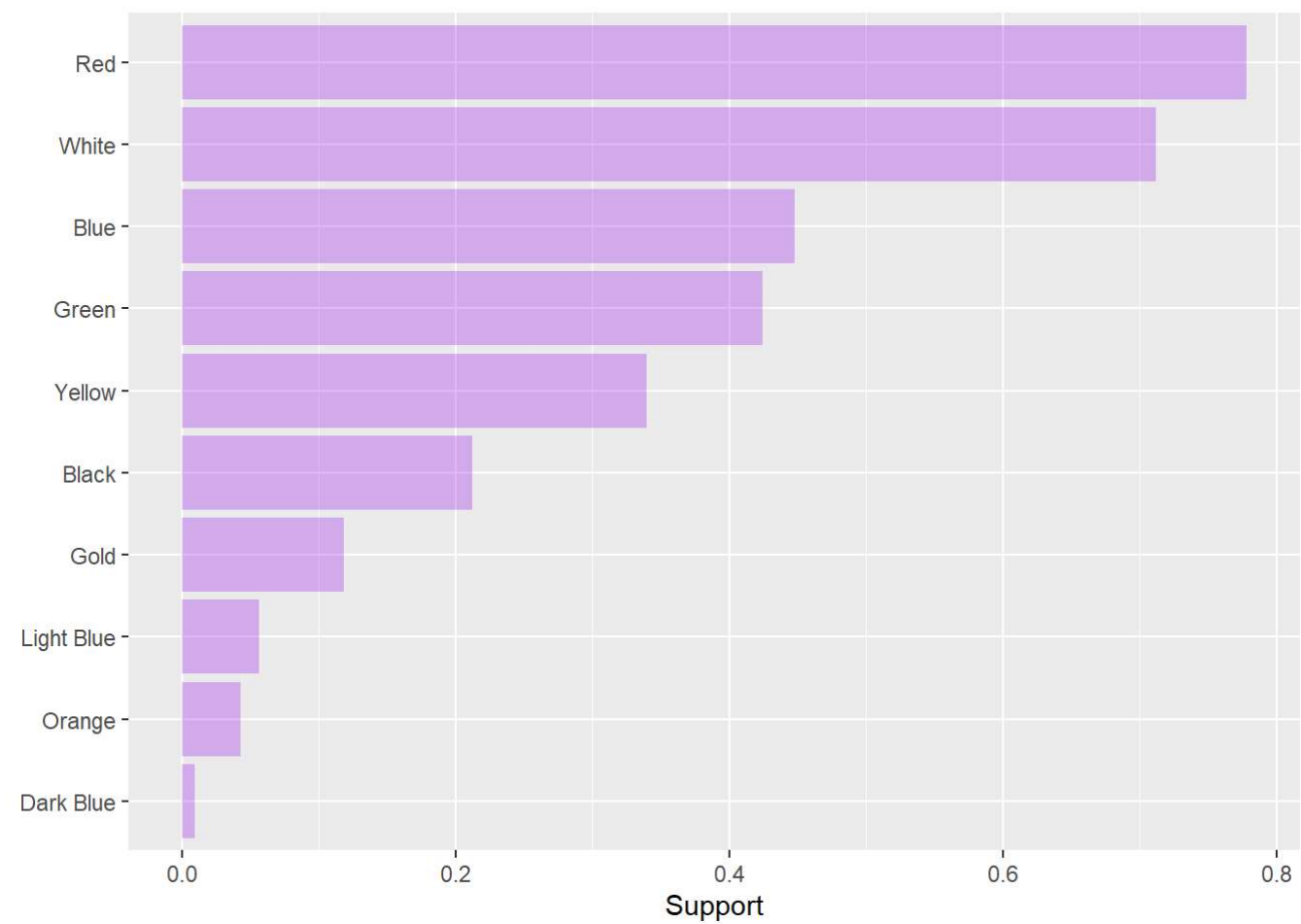
What are the top three most commonly used colours?

```
# Step1: Item frequencies
itemFrequencyPlot(flags, topN = 25, type = "relative")
```



```
# Step 2: Item frequencies of flag colours
flag_freq <- flags |>
  itemFrequency() |>
  enframe(name = "colour", value = "support") |>
  arrange(desc(support)) |>
  slice_head(n = 10)

ggplot(flag_freq, aes(x = support, y = fct_reorder(colour, support))) +
  geom_col(fill="purple",alpha=0.3) +
  labs(x = "Support", y = NULL)
```



- From the two plots above, we can conclude that:
- 1. Red is the most frequently used colour, which likely reflects histories of armed struggle and revolution;
 - 2. White is the second most commonly used colour, serving as a classic symbol of peace and purity;
 - 3. Blue ranks third, evoking themes of freedom and open skies.

4.2 Number of rules

Consider only rules where at least 6 countries have used the same colour combination. Use a confidence cutoff of `confidence = 0.5` .
How many rules are found?

```
# Mining rules with Apriori
flag_rules <- apriori(
  flags,
  parameter = list(support = 6/length(flags), confidence = 0.5, minlen = 1, maxlen = 4),
  control = list(verbose = FALSE)
)

flag_rules
```

```
## set of 64 rules
```

```
length(flag_rules)
```

```
## [1] 64
```

Under the two conditions, at least 6 countries sharing the same colour combination and a confidence of at least 0.5.

the algorithm picked up 64 meaningful colour pairing patterns from the flag dataset.

4.3 Highest support rules

Find the 5 rules with the highest support.

```
flag_rules |>
  sort(by = "support") |>
  head(n = 5) |>
  inspect()
```

```
##      lhs      rhs      support  confidence coverage  lift    count
## [1] {}      => {Red}    0.7783019 0.7783019  1.0000000 1.000000 165
## [2] {}      => {White} 0.7122642 0.7122642  1.0000000 1.000000 151
## [3] {White} => {Red}    0.5566038 0.7814570  0.7122642 1.004054 118
## [4] {Red}   => {White} 0.5566038 0.7151515  0.7783019 1.004054 118
## [5] {Blue}  => {White} 0.3349057 0.7473684  0.4481132 1.049285  71
```

Take the third rule in this list and explain:

- 1. what the rule itself means, that is, what `itemset => itemset` represents
- 2. the support value
- 3. the confidence value
- 4. the lift value

The third rule can be explained as follows:

- 1. `{White} => {Red}` means that flags which use white also tend to use red.
- 2. A support of 0.56 tells us that around 56% of all flags in the dataset contain both white and red at the same time.
- 3. A confidence of 0.78 means that among all flags that use white, 78% of them also use red.
- 4. A lift of 1.004, while technically above 1, is so close to 1 that white and red are essentially independent of each other statistically, knowing a flag has white tells you almost nothing extra about whether it also has red.

4.4 Lift greater than 2

Find the rules for which lift is greater than 2.

How many are there? Explain what these rules tell you.

```
# Find rules with lift greater than 2
lift_rules <- flag_rules |>
  subset(lift > 2) |>
  sort(by = "lift") |>
  inspect()
```

```
##      lhs      rhs      support  confidence coverage  lift
## [1] {Red, White, Yellow} => {Black} 0.05660377 0.5454545  0.1037736 2.569697
##      count
## [1] 12
```

There is only 1 rule with a lift greater than 2:

- 1. `{Red, White, Yellow} => {Black}` means that flags which use red, white, and yellow also tend to use black.
- 2. A support of 0.057 tells us that around 5.7% of all flags in the dataset contain red, white, yellow, and black at the same time.
- 3. A confidence of 0.55 means that among all flags that use red, white, and yellow, 55% of them also use black.
- 4. A lift of 2.57, which is well above 1, tells us that red, white, yellow, and black genuinely tend to appear together, flags that already use red, white, and yellow are about 2.57 times more likely to also include black than we would expect by chance.

4.5 Applied flag recommendation

Suppose a new island country declares independence. The leaders have decided that the flag must include green as a prominent colour.

Use the association rules to recommend one additional colour for the flag.

To keep the recommendation simple, consider rules with `Green` on the left-hand side and a single colour on the right-hand side. Which additional colour would you recommend, and why?

Your answer should refer to support, confidence, lift, and the colour frequencies where appropriate.

```
# Find rules with Green on the left-hand side
green_rules <- flag_rules |>
  subset(lhs %in% "Green" & lift > 1.1)

green_rules |>
  sort(by = "lift") |>
  inspect()
```

##	lhs	rhs	support	confidence	coverage
## [1]	{Blue, Green, Red}	=> {Yellow}	0.04245283	0.6000000	0.07075472
## [2]	{Blue, Green}	=> {Yellow}	0.06132075	0.5652174	0.10849057
## [3]	{Black, Green, White}	=> {Red}	0.08018868	0.9444444	0.08490566
## [4]	{Black, Green, Yellow}	=> {Red}	0.04716981	0.9090909	0.05188679
## [5]	{Black, Green}	=> {Red}	0.10849057	0.8846154	0.12264151
## [6]	{Green, White, Yellow}	=> {Red}	0.05660377	0.8571429	0.06603774
##	lift	count			
## [1]	1.766667	9			
## [2]	1.664251	13			
## [3]	1.213468	17			
## [4]	1.168044	10			
## [5]	1.136597	23			
## [6]	1.101299	12			

Based on the association rules, we recommend adding Yellow to the flag. Based on the association rules, we recommend adding Yellow to the flag.

1. Support: Around 6.1% of all flags use both green and yellow together, 13 countries enough to make this a credible recommendation.
2. Confidence: 56.5% of flags with green also use yellow.
3. Lift: At 1.664, this is the second highest lift among all green-related rules, meaning green and yellow genuinely tend to go together rather than just by chance.
4. Colour frequencies: Red appears frequently across many rules with high confidence, but red is already the most common colour that high confidence is just reflecting red's popularity, not a real strong link with green. Yellow is less common, so its lift is more trustworthy.

Although rule [1] has a higher lift of 1.767, it was not used here because its left-hand side already contains too many colours including red, which is itself a very common colour, making it a less clean basis for recommendation. Rule [2] keeps things simpler and more straightforward.